

Self-Calibrating Divergence Detection for Adaptive Best-of-N Reasoning

A Mechanism Layered on Exponential Smoothing for Compute-Efficient Chain Selection

Cole Cantrell | cole@paradigmbridge.tech | paradigmbridge.tech

Paper 20 of an Ongoing Series | github.com/HauntedKernel/power-metric

Version 2 — May 2026

Abstract

Best-of-N reasoning over multiple chains-of-thought is a standard test-time compute strategy, but naive implementations run all candidate chains to completion before selecting a winner. This work introduces a self-calibrating divergence detector that identifies when chain trajectories have meaningfully separated, paired with a hybrid disposition policy that hard-kills clearly failed chains and scaffolds borderline ones at a verifier-cost discount. The detector uses a z-scored gap statistic against a per-problem null distribution established during the shared-prefix grace period, eliminating the need for dataset-specific detection thresholds.

In static-label simulations on PRM800K, the mechanism reduces step-equivalent best-of-N compute by 22.8% at 99.6% winner accuracy, matching the data's achievable upper bound of 99.6% (100.0% of oracle). On Math-Shepherd, where auto-rollout labels produce a much lower information ceiling, the same architecture with identical parameters reduces compute by 13.6% at 58.4% winner accuracy — 96.4% of the dataset's 60.6% oracle. Across a 3×3 sensitivity sweep of the disposition parameters, winner accuracy is unchanged and compute saving varies by at most 3.3 percentage points, with zero killed-correct chains throughout. The mechanism is compute-cheap (negligible overhead relative to the inference it monitors), training-free, and saturates the achievable information ceiling on both datasets within sensitivity bounds.

The contribution is the mechanism, not the saving number: three layers of structure beyond exponential smoothing (per-problem null-calibrated detection, hybrid level-based disposition, literature-grounded scaffold cost) that together produce a deployable adaptive branching scheme with no learned components.

Note on this version

This is version 2 of Paper 20. The original version reported 29.4% PRM800K compute saving and 21.8% Math-Shepherd compute saving. Subsequent code review identified three correctness fixes: (i) prompt-grouping by SHA256 of full prompt rather than first-100-character truncation (which caused 1 spurious merge on PRM800K and 814 on Math-Shepherd); (ii) cost-accounting off-by-one (`prefix_steps = branch_step + 1`) for the shared prefix; (iii) deterministic problem selection via sorted-key order. Version 2 reports the corrected numbers and adds three baseline metrics (random, end-of-chain ranker, oracle) plus a 3×3 parameter sensitivity sweep. The mechanism, architecture, and parameters are unchanged. The previous version remains accessible on Zenodo for citation continuity.

1. Introduction

Test-time compute scaling through multi-chain reasoning is widely deployed: sample N candidate chains-of-thought, evaluate each, return the best. The cost of this strategy is linear in N , and existing reductions either truncate at fixed steps, prune by absolute reward thresholds, or rely on learned policies.

Each approach has known limits: fixed truncation cannot adapt to per-problem difficulty; absolute thresholds do not transfer across datasets with different signal distributions; learned policies require training data and add architectural dependencies.

We propose a self-calibrating mechanism that operates on top of any per-step health signal. The mechanism has three layers: (1) a level-based divergence detector that fires when the inter-chain gap exceeds the per-problem null distribution by a fixed number of standard deviations; (2) a hybrid disposition policy that hard-kills chains below an absolute health level and scaffolds the remainder against a verifier-cost discount; (3) a quantitative grounding for the discount in the speculative decoding literature. The mechanism requires no training, no dataset-specific detection threshold, and routes saving through structurally different paths depending on the chain-quality regime of each dataset.

In static-label simulations on PRM800K reasoning chains, the mechanism reduces step-equivalent best-of-N compute by 22.8% while preserving 99.6% winner accuracy and producing zero killed-correct chains. The 99.6% winner accuracy matches the dataset’s achievable oracle bound — the mechanism extracts essentially all available signal at the branch step. On Math-Shepherd, the same architecture reduces compute by 13.6% at 58.4% winner accuracy, where the oracle bound is 60.6% because 39.4% of Math-Shepherd problem groups contain no correct chain among their candidates. The mechanism reaches 96.4% of this lowered ceiling.

Contributions:

1. A self-calibrating divergence detector with per-problem null estimation, eliminating dataset-specific detection thresholds.
2. A hybrid disposition policy that routes between hard-kill and discounted scaffolding based on chain health, with the discount grounded in published speculative-decoding speedups.
3. Empirical demonstration on two datasets with structurally different chain-quality regimes, with a sensitivity sweep showing the operating point is not cherry-picked and an information-ceiling analysis showing the mechanism saturates the achievable bound on both datasets.

2. Background: A Health Signal for Reasoning Chains

We assume access to a per-step correctness signal for each reasoning chain. In our experiments this comes from process reward model labels (PRM800K) or auto-rollout verification labels (Math-Shepherd), but the mechanism is agnostic to the source.

For a chain producing scores $s(1), s(2), \dots, s(t)$, we track three quantities with exponential smoothing:

$$E[R](t) = (1-\alpha) \cdot E[R](t-1) + \alpha \cdot s(t)$$

$$E(t) = s(t) / E[R](t-1)$$

$$W(t) = \beta \cdot 1[E(t) \geq 1] + (1-\beta) \cdot W(t-1)$$

$$P(t) = \exp(-\lambda) \cdot P(t-1) + (1-\exp(-\lambda)) \cdot E(t) \cdot W(t)$$

Where $E[R]$ is the adaptive baseline of expected step rewards, $E(t)$ is the efficiency ratio of current step against expected, $W(t)$ is the EWMA win rate of efficiency exceeding parity, and $P(t)$ is the smoothed health metric. Default parameters: $\alpha = 0.5$, $\beta = 2/(\text{span}+1)$ with $\text{span} = 3$, $\lambda = 0.5$. We initialize $E[R]$ at 0.5 with an ϵ -floor on division to avoid blowup at chains with early-zero rewards.

Prior work (Cantrell 2026, Paper 18) established that this signal evaluated at chain end ranks reasoning chains at correlation $r > 0.9999$ against ground-truth correctness on PRM800K. The present work asks

whether the signal supports online decisions during chain generation, before chains complete.

A natural concern is that the smoothing structure is functionally similar to standard exponential smoothing methods such as ETS, Holt-Winters, and simple exponential smoothing (Hyndman & Athanasopoulos 2021). The point of the present work is that the smoothed signal is the substrate, not the contribution. The contribution is the layer of mechanism that operates on top of the smoothed signal: divergence detection, hybrid disposition, and literature-grounded cost accounting.

3. The Mechanism

For a problem with N candidate chains, we maintain N parallel $P(t)$ trajectories. The mechanism has three layers operating on these trajectories.

3.1 Self-Calibrating Divergence Detection

Define the inter-chain gap at step t as the difference between the maximum and minimum $P(t)$ across live chains:

$$\text{gap}(t) = \max_i P_i(t) - \min_i P_i(t)$$

For the first G steps, all chains share a prefix and should produce nearly identical $P(t)$ trajectories. We use this grace window to estimate a per-problem null distribution for the gap:

$$\mu_{\text{gap}} = \text{mean}(\text{gap}(0..G-1)), \quad \sigma_{\text{gap}} = \max(\text{std}(\text{gap}(0..G-1)), \sigma_{\min})$$

Where $\sigma_{\min}$ is a numerical floor preventing division by zero on identical grace gaps. After the grace period, at each step t we compute the z-score:

$$z(t) = (\text{gap}(t) - \mu_{\text{gap}}) / \sigma_{\text{gap}}$$

The detector fires at the first step where $z(t) > k$. We use $k = 3.0$ (a conservative three-sigma heuristic; because the grace window is short, this is a scale-normalized trigger rather than a formal Gaussian hypothesis test) and $G = 3$ in all experiments.

Why this is self-calibrating: PRM800K chains share clean prefixes that produce $\sigma_{\text{gap}} \approx 0.005$ (hitting the floor), so any meaningful divergence triggers. Math-Shepherd chains have noisier prefixes that produce $\sigma_{\text{gap}} \approx 0.090$, so the detector requires a correspondingly larger separation to fire. Same parameter k , dataset-appropriate sensitivity. No detection-threshold tuning is required. (The disposition parameters L_{kill} and d remain fixed operating choices; see Section 4.4.)

3.2 Hybrid Disposition

When the detector fires at step t^* , we identify the candidate winner as the chain with the highest $P(t^*)$ and apply a per-chain disposition rule to each non-winner:

- If $P_i(t^*) < L_{\text{kill}}$: the chain is unambiguously dead. Hard-kill (no further compute spent on chain i).
- Otherwise: the chain is borderline. Pause and resume against the winner's final $P(t)$ at scaffold-discount cost.

We use $L_{\text{kill}} = 0.40$ in all experiments. The asymmetry between detection (z-score, relative) and disposition (absolute level) is intentional: they answer different questions. Detection asks “did the inter-chain situation change?”, which is fundamentally a relative judgment against a null. Disposition asks “is this specific chain dead enough that further compute is wasted?”, which is fundamentally an absolute judgment against accumulated multi-step evidence. The post-grace context provides enough evidence per chain for absolute reasoning to be reliable.

3.3 Scaffolding Discount

A paused chain that survives the kill threshold can be resumed by evaluating its remaining steps against the winner’s final $P(t)$ as a fixed denominator (offensive variant of the efficiency ratio). The cost of this verifier-style resumption is empirically smaller than full generation.

We set the scaffolding discount $d = 0.30$, motivated by the speculative decoding literature: Leviathan et al. (2023) report 2-3× wall-clock speedups on draft-and-verify decoding, implying verifier cost is roughly 0.33-0.50 of generation cost. Chen et al. (2023) report similar ratios. $d = 0.30$ is the aggressive end of this published range. We do not claim a novel speedup mechanism; we claim that paused-branch resumption is structurally a verification task and inherits its cost ratio. This is a cost model, not an empirically measured speedup — we report step-equivalent compute throughout.

3.4 Cost Accounting

For a problem group with N chains, branch step t^* (zero-based trajectory index), and prefix length $p = t^* + 1$ (number of steps consumed before branching), the total step-equivalent compute cost under our cost model is:

$$C_{\text{adaptive}} = N \cdot p + (\text{len}(\text{winner}) - p) + d \cdot \sum_{i \in S} (\text{len}_i - p)$$

where S is the set of non-winner chains above the kill threshold (scaffolded), and chains below the kill threshold contribute zero further cost. Compared to standard best-of- N compute $C_{\text{std}} = \sum_i \text{len}_i$, the compute reduction is $C_{\text{std}} - C_{\text{adaptive}}$, which decomposes into three sources: (1) early branching saves shared compute on chains beyond t^* , (2) hard-kills save the full remaining cost on dead chains, (3) scaffolded resumption saves $(1-d)$ of the remaining cost on borderline chains.

4. Empirical Results

4.1 Datasets and Simulation Protocol

We evaluate on two reasoning chain datasets with structurally different verification regimes. PRM800K (Lightman et al. 2023) provides human-annotated step-level correctness labels for mathematical reasoning chains. We use 30,298 chains across 626 multi-chain problems, simulating the first 500 problem groups in sorted-key order. Math-Shepherd (Wang et al. 2024) provides auto-generated step-level labels derived from rollout-based verification on 20,259 sampled chains across 7,042 multi-chain problems.

Simulation protocol. All experiments are simulations over recorded step-label trajectories. We do not measure wall-clock inference time or execute live LLM generation. Compute is reported in step-equivalent units. Scaffolded continuation is modeled using a fixed discount $d = 0.30$, motivated by verifier-style decoding costs (Section 3.3), rather than directly benchmarked. Grouping uses SHA256 of the full prompt to avoid spurious merges; problem order is sorted-key deterministic; cost accounting uses $\text{prefix_steps} = \text{branch_step} + 1$ to correctly account for the zero-based trajectory index.

Definitions. A chain is "all-correct" if every step label is True under the dataset’s step labels. Winner accuracy is the fraction of problem groups in which the chain selected by the mechanism is all-correct. Killed-but-correct is defined conservatively as a killed chain whose full recorded label sequence is all-correct under the dataset labels; this is a stricter diagnostic than final-answer correctness. Oracle accuracy is the fraction of problem groups in which at least one candidate chain is all-correct (the maximum achievable by any selection strategy on this candidate pool).

4.2 Headline Results

Table 1 reports the operating-point results on both datasets at the chosen parameters ($k = 3.0$, $G = 3$, $L_{\text{kill}} = 0.40$, $d = 0.30$).

Metric	PRM800K	Math-Shepherd
Multi-chain problem groups	626	7,042
Groups simulated	500	500
Mean chains per problem	50.8	2.9
Detector fire rate	95.0%	26.8%
Mean branch step (fired)	6.0	4.3
Mean chains hard-killed	0.0	0.7
Mean chains scaffolded	43.4	0.5
Random chain accuracy (baseline)	26.0%	34.6%
End-of-chain P(t) ranker	99.6%	60.6%
Oracle (any-chain correct)	99.6%	60.6%
Winner accuracy (ours)	99.6%	58.4%
% of oracle	100.0%	96.4%
Killed-but-correct rate	0.0%	0.0%
Compute saving	22.8%	13.6%

Table 1. Headline results under identical parameters across both datasets. The mechanism reaches 100.0% of the PRM800K oracle and 96.4% of the Math-Shepherd oracle. The Math-Shepherd oracle is lower (60.6%) because 39.4% of Math-Shepherd problem groups contain no all-correct chain in the candidate pool; this is a property of the data generation process, not the mechanism. Zero killed-correct chains across both datasets.

4.3 Information-Ceiling Interpretation

The % of oracle metric is the most informative way to read the winner-accuracy numbers. On PRM800K, with mean 50.8 chains per problem and clean human-annotated labels, the oracle is 99.6%: almost every problem group contains at least one all-correct chain. The mechanism finds it 99.6% of the time — it saturates the achievable ceiling. The end-of-chain P(t) ranker (the upper bound for online ranking under full information) achieves the same 99.6%, meaning the mechanism extracts essentially all available signal at the branch step despite operating on partial-information traces.

On Math-Shepherd, mean 2.9 chains per problem and noisy auto-rollout labels combine to give an oracle of 60.6%: in nearly two of every five problem groups, no candidate chain is all-correct. The end-of-chain P(t) ranker also reaches 60.6%, indicating P(t) is matching the oracle even with full information. Our online ranker at the branch step achieves 58.4%, within 2.2 points of the oracle (96.4% of oracle). The bottleneck on Math-Shepherd is the candidate pool, not the ranking mechanism. Better Math-Shepherd performance requires generating better candidate chains, not better ranking among the existing ones.

4.4 Sensitivity Sweep

A natural reviewer concern is that the chosen disposition parameters ($L_{\text{kill}} = 0.40$, $d = 0.30$) were selected post hoc to maximize the headline numbers. To address this, we report a 3×3 sensitivity sweep across nine combinations of $L_{\text{kill}} \in \{0.30, 0.40, 0.50\}$ and $d \in \{0.25, 0.30, 0.35\}$, with $k = 3.0$ and $G = 3$ held fixed.

L_{kill}	d	PRM Save	PRM %Or	MS Save	MS %Or
0.30	0.25	24.4%	100.0%	13.9%	96.4%
0.30	0.30	22.8%	100.0%	13.6%	96.4%
0.30	0.35	21.2%	100.0%	13.3%	96.4%
0.40	0.25	24.4%	100.0%	13.9%	96.4%
0.40	0.30	22.8%	100.0%	13.6%	96.4%
0.40	0.35	21.2%	100.0%	13.3%	96.4%
0.50	0.25	24.4%	100.0%	14.2%	96.4%
0.50	0.30	22.8%	100.0%	14.0%	96.4%
0.50	0.35	21.2%	100.0%	13.8%	96.4%

Table 2. Sensitivity sweep across the disposition parameters. PRM800K compute saving varies from 21.2% to 24.4% (range 3.3 pp); Math-Shepherd from 13.3% to 14.2% (range 0.9 pp). Winner accuracy and % of oracle are unchanged across the entire grid in both datasets. Killed-but-correct stays at 0.0% throughout. The highlighted row is the chosen operating point.

Three findings from this sweep are worth noting. **First**, winner accuracy is fully determined by the detection and ranking layers and is invariant to the disposition parameters. The kill threshold and scaffold discount only affect compute cost, not correctness. **Second**, on PRM800K, defensive $P(t)$ almost never drops below 0.30 at the branch step, so raising L_{kill} from 0.30 to 0.50 does not engage additional kills; the parameter is inert on clean data. **Third**, killed-but-correct rate remains zero across all 18 cells (9 per dataset). The chosen operating point is therefore not cherry-picked: no other combination in this neighborhood produces a materially different outcome.

5. Discussion: Information-Availability and the Pool Bottleneck

The Math-Shepherd result was initially read as a failure of online ranking under noisy verification labels. The introduction of the oracle and end-of-chain baselines reframes this interpretation. The end-of-chain $P(t)$ ranker on Math-Shepherd achieves 60.6% — exactly the oracle — meaning $P(t)$ ranks at the data’s achievable maximum even with full trajectory information. The online ranker at the branch step achieves 58.4%, only 2.2 points below this ceiling.

The bottleneck on Math-Shepherd is therefore not the ranker, the detector, or the partial-information regime. It is the candidate pool. With mean 2.9 chains per problem and a per-chain correctness rate near 30%, 39.4% of problem groups contain no all-correct chain to select. No ranking strategy — ours, end-of-chain, learned PRM, or any other — can recover a correct answer when none is present in the pool.

This reframes the temporal information-availability question. PRM800K and Math-Shepherd both saturate their achievable bounds; the bounds themselves differ because the chain-generation processes that produced the candidate pools differ. PRM800K’s human-curated multi-attempt pool has a near-perfect oracle; Math-Shepherd’s sparse auto-rollout pool has a much lower one. A practitioner deploying the mechanism on a noisy auto-verifier pipeline should expect compute saving but should also generate enough candidate chains per problem to lift the pool oracle into a useful range; ranking quality is bounded by what the pool contains.

On the “temporal information-availability bound” framing. We use “bound” informally here. A formal treatment would require estimating verifier noise and deriving the minimum horizon at which inter-chain quality gaps dominate integrated label noise. We have not done that derivation in this paper. What we have shown empirically is that on both datasets the mechanism reaches the achievable ceiling within sensitivity bounds, and that the ceiling itself is determined by the candidate pool rather than by the ranking horizon.

6. Related Work

Best-of-N reasoning. Process reward models (Lightman et al. 2023; Wang et al. 2024) select among multiple chains-of-thought using learned step-level scorers. Our mechanism is complementary: it accepts any per-step signal and adds a compute-saving layer, including the option to use process reward model outputs directly as the $s(t)$ in our formulation.

Speculative decoding. Leviathan et al. (2023) and Chen et al. (2023) achieve 2-3× speedups by using a small draft model to propose tokens and a larger target model to verify. Our scaffolding discount $d = 0.30$ is grounded in this literature: paused-branch resumption is structurally a verification task, and we apply published verifier-cost ratios.

Tree search and MCTS for reasoning. Tree-of-Thoughts (Yao et al. 2023) and related approaches selectively expand reasoning trees using application-specific value functions, often learned. Our mechanism is non-learned and operates on flat best-of-N pools rather than tree expansions, making it applicable where training a value function is impractical.

Time-series change-point detection. Our divergence detector is structurally a sequential change-point test on a dimensionless statistic (Page 1954; Basseville & Nikiforov 1993). Compared to CUSUM, our detector uses a simpler level-based z-score that avoids the need for a hypothesized post-change mean, at the cost of some theoretical optimality.

Exponential smoothing literature. ETS, Holt-Winters, and SES (Hyndman & Athanasopoulos 2021) provide the smoothing substrate for $P(t)$. The mechanism layered on top is what distinguishes our approach: divergence detection, hybrid disposition, and verifier-cost grounding are not part of the smoothing literature.

7. Limitations

1. Static-label simulation, not live inference. We evaluate on recorded step-label trajectories. We do not measure wall-clock inference time or execute live LLM generation. Compute is reported in step-equivalent units under a cost model. Production deployment would require integrating the mechanism with a streaming inference engine and validating that the scaffolding-discount assumption matches the actual cost of resumed-branch verification on the target architecture.

2. Cost model rather than measured speedup. The scaffolding discount $d = 0.30$ is borrowed from speculative-decoding literature. We do not empirically measure verifier cost in our simulations. The cost-model framing is honest about this throughout.

3. In-sample evaluation on a single dataset version. We evaluate on PRM800K (trl-lib/prm800k) and Math-Shepherd as they exist; we do not perform cross-validation or held-out evaluation. The % of oracle metric protects against overclaiming because the ceiling is dataset-specific, but cross-validation would strengthen the operating-point claims.

4. The Math-Shepherd candidate pool is sparse. With mean 2.9 chains per problem, Math-Shepherd is a thinner pool than realistic best-of-N deployments would typically use. The reported saving numbers there are conservative because there is less compute to save per group.

5. Parameter selection grounded in citation rather than formal sensitivity analysis. $d = 0.30$ is defended from speculative decoding literature; $k = 3.0$ is the standard three-sigma heuristic; $G = 3$ is the smallest grace window we found reasonable for estimating both a mean and a variance while preserving early branching. The 3×3 sweep in Section 4.4 demonstrates robustness to disposition parameters but does not sweep k or G .

6. The temporal information-availability framing is informal. We do not derive a formal noise-rate bound on partial-information ranking. The empirical observation is that the mechanism reaches the candidate-pool oracle on both datasets within sensitivity bounds; the formal characterization of when this holds is left for future work.

8. Conclusion

A health signal computed via exponential smoothing supports adaptive best-of- N reasoning when paired with a self-calibrating divergence detector and a hybrid disposition policy. In static-label simulations on PRM800K, the mechanism reduces step-equivalent compute by 22.8% at 99.6% winner accuracy — 100% of the achievable oracle — with the saving mathematically grounded in three layers: per-problem null calibration breaks dataset-specificity of detection, level-based hard-kill disposes of obviously failed chains, and verifier-cost-grounded scaffolding handles borderline cases. The same architecture applies without modification to Math-Shepherd, where it delivers 13.6% saving at 58.4% accuracy — 96.4% of the lower oracle ceiling produced by the sparser auto-rollout candidate pool. A 3×3 sensitivity sweep confirms the operating point is not cherry-picked: across 18 parameter combinations, winner accuracy is invariant, compute saving varies by at most 3.3 pp, and zero killed-correct chains occur in any cell.

The mechanism is the contribution; the saving numbers are the consequence. The information-ceiling analysis demonstrates that the mechanism saturates the achievable bound on both datasets, and that the bottleneck on noisy data is the candidate pool rather than the ranker. The compute overhead of the mechanism itself is negligible relative to the inference it monitors, making it suitable for deployment as a lightweight, training-free layer on top of existing best-of- N pipelines.

References

- Basseville, M., & Nikiforov, I. V. (1993). *Detection of Abrupt Changes: Theory and Application*. Prentice Hall.
- Cantrell, C. (2026). *The Power Metric: Adaptive Compute Allocation via Stochastic Health Signals* (Paper series, including Paper 18 on chain selection at $r > 0.9999$). github.com/HauntedKernel/power-metric.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., & Jumper, J. (2023). Accelerating large language model decoding with speculative sampling. *arXiv preprint*.
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.
- Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast inference from transformers via speculative decoding. *ICML 2023*.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2023). Let’s verify step by step. *arXiv:2305.20050*.
- Page, E. S. (1954). Continuous inspection schemes. *Biometrika*, 41(1/2), 100–115.
- Wang, P., Li, L., Shao, Z., Xu, R. X., Dai, D., Li, Y., Chen, D., Wu, Y., & Sui, Z. (2024). Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. *ACL 2024*.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of Thoughts: Deliberate problem solving with large language models. *NeurIPS 2023*.